Maestría en Sistemas Embebidos
Programación y Procesamiento de Datos
5A. Práctica de hilos en Python

Alumno: Claudio Iván Esparza Castañeda
Profesor: Luis Alberto Balam Guzmán

Objetivo

Lo que hizo Arduino por la cultura maker fue fundamental para democratizar la tecnología. Por otro lado, otro impulso se dio a través del uso de Python por la misma comunidad (junto a su creador, Guido Van Rossum). En algún punto estas dos fuerzas convergieron para llegar a micropython, que es un proyecto que empuja hacia esta democratización para el tipo de usuario que en su día a día no tiene tiempo de aprender todo acerca del hardware, pero busca realizar proyectos de fin de semana, o bien el profesional o el estudiante que requiere un prototipado rápido para el proyecto de fin de semana. Después del Arduino, han aparecido diferentes soluciones con distintos controladores. Así es como llegamos a la Raspberry Pi Pico, que se puede programar a través de bare-metal, con RTOS (FreeRTOS, Zephyr, etc.), o micropython.
Con la imposibilidad que representa tener que procesar grandes cantidades de información en un medio con pocos recursos, refiriéndonos en este caso a la Raspberry Pi Pico, surge la necesidad de contar con sistemas híbridos: control en el sistema embebido, y procesamiento en la computadora. En sistemas modernos, se cuenta con controladores y procesadores con múltiples núcleos. Eso, además de arquitecturas basadas en pipeline permiten manejar múltiples operaciones en un solo chip, empleando distintas operaciones.

Descipción de la solución

Se sigue la estructura propuesta para el proyecto. Hay un único cambio realizado en el archivo `generador_senial.py` fue renombrarlo a `main.py` e incluirlo en la carpeta `pico/`, Este cambio se realizó con el objetivo de poder usar el puerto serial sin que se bloqueé el puerto serial por el REPL. Por otro lado, se dividieron responsabilidades a través de las distintas partes requeridas. Por un lado, el archivo `adquisicion.py` contiene la información relacionada al primer hilo. El archivo `procesamiento.py` se encuentra el segundo hilo, donde se realiza el procesamiento de la señal. Por último, el archivo `registro.py` contiene el tercer hilo, y guarda el log en un csv.
Se sigue usando el archivo `funciones.py` de tareas anteriores, con el objetivo de hacer algunas validaciones. El archivo `main.py` se encarga de ejecutar todo lo anterior en el orden dado.

Metodología

El tratamiento de las señales se realiza con SciPy, al emplear las funciones definidas en este módulo. Se emplea SciPy para calcular los valores para el procesamiento de señal, como el RMS, promedio, desviación estándar.
Se emplea Serial Studio para el análisis de la transmisión serial con la tarjeta. Posteriormente se trabaja para codificar la comunicaicón en el programa de python.

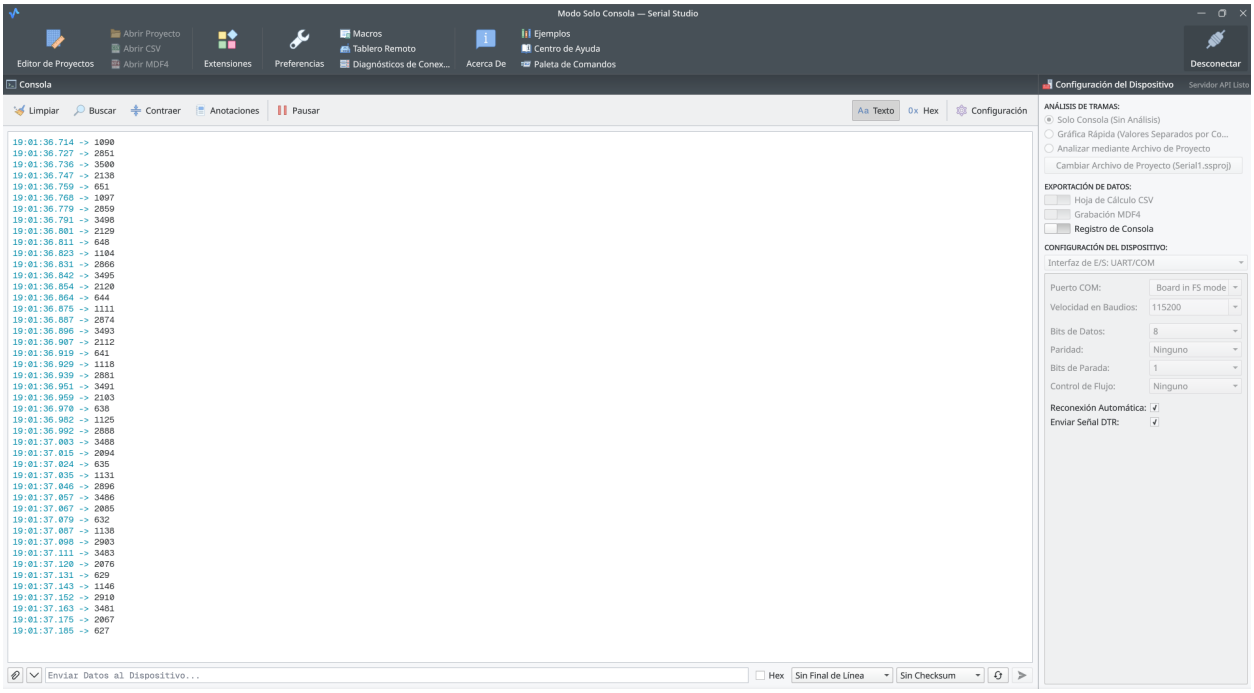


Figure 1: Serial studio con la transmisión serial

```
~/Documentos/Escuela/INFOTEC/Programación y procesamiento de datos/Programacion_y_Procesamiento_de_Datos_Infotec/5A main* 12s
❯ python main.py
=== Monitoreo de vibración - Raspberry Pi Pico ===

[Adquisición] Conectando a /dev/ttyACM1 @ 115200 baudios...
Sistema corriendo. Presiona Ctrl+C para detenerlo.

[Adquisición] Conectado. Leyendo muestras...
[Procesamiento] 2026-10-17 18:54:49 | Prom=2047.34 Max=3428 Min=570 RMS=2305.8 Std=1060.72 | Frecuencia dominante ≈ 19.81 Hz (fs≈99.06 Hz)
[Registro] Bloque guardado en 'resultados/vibracion.csv'.
[Procesamiento] 2026-10-17 18:54:51 | Prom=2047.4 Max=3395 Min=558 RMS=2305.86 Std=1060.73 | Frecuencia dominante ≈ 19.63 Hz (fs≈98.14 Hz)
[Registro] Bloque guardado en 'resultados/vibracion.csv'.
```

Figure 2: Inicio de ejecución

```
~/Documentos/Escuela/INFOTEC/Programación y procesamiento de datos/Programacion_y_Procesamiento_de_Datos_Infotec/5A main* 12s
❯ python main.py
[Procesamiento] 2026-10-17 18:55:09 | Prom=2047.45 Max=3541 Min=683 RMS=2306.04 Std=1061.01 | Frecuencia dominante ≈ 39.56 Hz (fs≈98.91 Hz)
[Registro] Bloque guardado en 'resultados/vibracion.csv'.
[Procesamiento] 2026-10-17 18:55:12 | Prom=2047.53 Max=3525 Min=656 RMS=2305.88 Std=1060.54 | Frecuencia dominante ≈ 39.29 Hz (fs≈98.22 Hz)
[Registro] Bloque guardado en 'resultados/vibracion.csv'.
[Procesamiento] 2026-10-17 18:55:14 | Prom=2047.58 Max=3547 Min=704 RMS=2305.79 Std=1060.23 | Frecuencia dominante ≈ 39.19 Hz (fs≈97.96 Hz)
[Registro] Bloque guardado en 'resultados/vibracion.csv'.
[Procesamiento] 2026-10-17 18:55:17 | Prom=2047.64 Max=3548 Min=750 RMS=2305.86 Std=1060.27 | Frecuencia dominante ≈ 39.67 Hz (fs≈99.18 Hz)
[Registro] Bloque guardado en 'resultados/vibracion.csv'.
[Procesamiento] 2026-10-17 18:55:20 | Prom=2047.7 Max=3539 Min=659 RMS=2305.98 Std=1060.4 | Frecuencia dominante ≈ 39.53 Hz (fs≈98.83 Hz)
[Registro] Bloque guardado en 'resultados/vibracion.csv'.
```

Figure 3: Ejecución contínua

```
~/Documentos/Escuela/INFOTEC/Programación y procesamiento de datos/Programacion_y_Procesamiento_de_Datos_Infotec/5A main* 12s
❯ python main.py
[Procesamiento] 2026-10-17 18:56:36 | Prom=2046.82 Max=3514 Min=551 RMS=2305.4 Std=1060.86 | Frecuencia dominante ≈ 40.0 Hz (fs≈100.0 Hz)
[Registro] Bloque guardado en 'resultados/vibracion.csv'.
[Procesamiento] 2026-10-17 18:56:39 | Prom=2047.15 Max=3547 Min=682 RMS=2306.03 Std=1061.58 | Frecuencia dominante ≈ 40.0 Hz (fs≈100.0 Hz)
[Registro] Bloque guardado en 'resultados/vibracion.csv'.
[Procesamiento] 2026-10-17 18:56:41 | Prom=2047.53 Max=3531 Min=560 RMS=2306.5 Std=1061.88 | Frecuencia dominante ≈ 40.0 Hz (fs≈100.0 Hz)
[Registro] Bloque guardado en 'resultados/vibracion.csv'.
^C
Deteniendo hilos...
[Adquisición] Finalizada. Muestras descartadas (no numéricas): 0
[Procesamiento] Finalizado.
```

Figure 4: Ejecución contínua

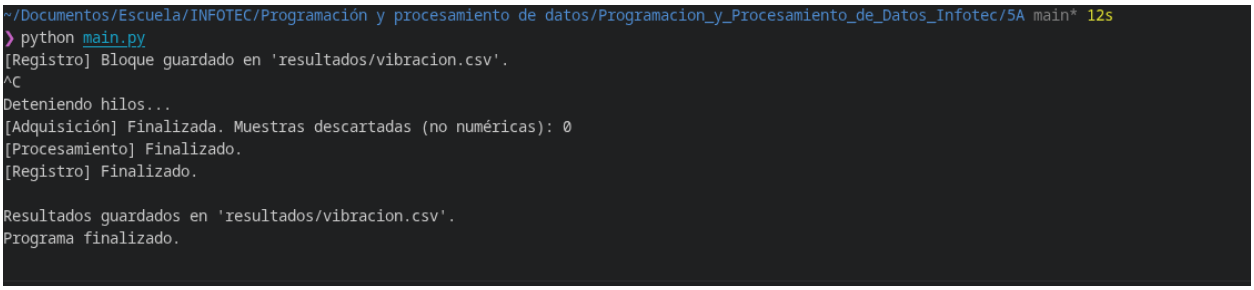


Figure 5: Corte de la ejecución del programa

Capturas de ejecución

Conclusiones

- **¿Qué problema se presentaría si todo fuera presencial?** El programa tendría que leer, procesar y duardar en ese orden, y mientras hace una cosa no puede hacer las otras. El retardo provocaría el bloqueo del puerto serial.
- **¿Qué ventajas aportan los hilos en adquisión de datos?** La existencia de la concurrencia real de tareas permite que la lectura no se detenga. Permiten aprovechar los tiempos de espera que existen entre la lectura serial y la escritura a disco, pues ambas son operaciones de entrada y salida. Permite aguardar el Ctrl+C mientras trabaja.
- **¿Por qué separar adquisión y procesamiento?** La adquisión debe ocurrir al ritmo del sensor. El procesamiento es de tiempo flexible, lo que puede tardar más o menos según la carga. Si ambos van juntos, el más lento dicta el ritmo del otro.
- **¿En qué aplicaciones reales se usan arquitecturas similares?** Monitoreo de entorno industriales, automotriz que incorpora la adquisión de de sensores separada del procesamiento y la comunicación. Telemetría y robótica, entre otras aplicaciones.

La carpeta en el repositorio de este proyecto es: Programación y procesamiento de Datos.